

Chuyên đề: Thuật Toán Mo (Mo's Algorithm & Square Root Query Decomposition)

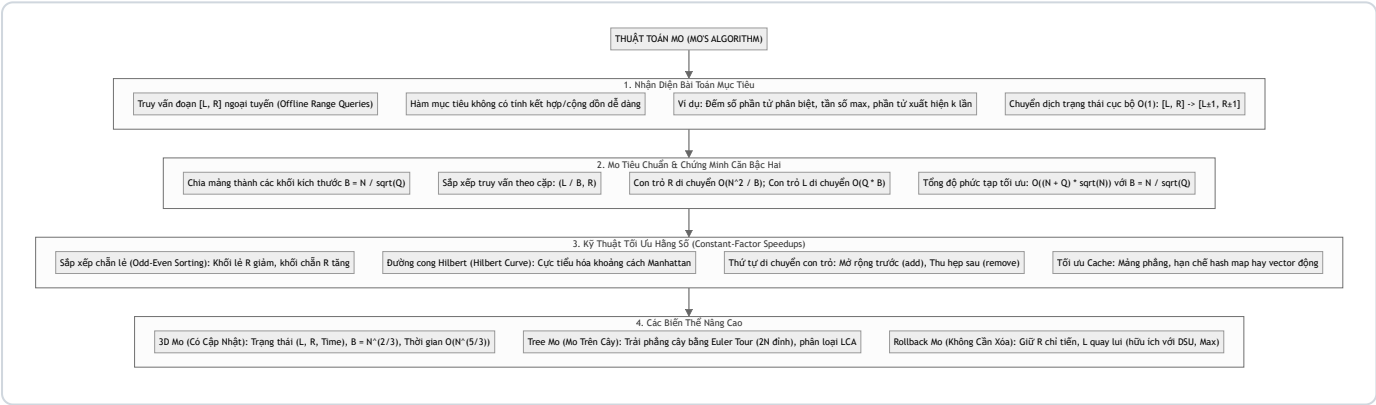
(Offline Range Queries, Odd-Even Sorting, 3D Mo with Updates, Mo on Trees via Euler Tour, Rollback Mo without Deletions, Hilbert Curve)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy Toàn cảnh (Mindmap)



1. Đặt Vấn Đề & Bản Chất Thuật Toán Mo

1.1. Bối cảnh xuất hiện

Trong lập trình thi đấu, dạng bài toán xử lý truy vấn đoạn trên mảng $A[1 \dots N]$ là dạng bài kinh điển:

$$\text{Query}(L, R) \rightarrow \text{kết quả trên đoạn } [L, R]$$

Nếu bài toán có tính chất **cộng dồn** hoặc **kết hợp** (như tính tổng, tìm $\min / \max$, gcd), ta có thể dễ dàng giải quyết trong thời gian $O(\log N)$ hoặc $O(1)$ trực tuyến bằng **Segment Tree**, **Fenwick Tree** hoặc **Sparse Table**.

Tuy nhiên, có một lớp bài toán lớn mà thông tin thống kê kê trên $[L, R]$ **không thể gộp trực tiếp** từ hai nửa trái phải $[L, M]$ và $[M + 1, R]$ một cách độc lập trong $O(1)$. Ví dụ:

- Đếm số lượng phần tử phân biệt trong đoạn $[L, R]$ (SPOJ DQUERY).
- Tính $\sum_x (\text{cnt}[x])^2 \cdot x$ với $\text{cnt}[x]$ là số lần xuất hiện của x trong đoạn $[L, R]$ (Codeforces 86D - Powerful array).

- Tìm giá trị nhỏ nhất chưa xuất hiện (MEX) trong đoạn $[L, R]$.
- Đếm số giá trị xuất hiện đúng k lần trong đoạn $[L, R]$.

1.2. Ý tưởng cốt lõi: Chuyển dịch hai con trỏ (Two-Pointers State Transition)

Giả sử ta đang duy trì trạng thái thống kê của một đoạn con $[currL, currR]$.

Nếu bài toán cho phép:

- Mở rộng đoạn sang phải thêm 1 phần tử ($currR \rightarrow currR + 1$) trong thời gian $O(1)$ thông qua hàm `add(currR + 1)`.
- Mở rộng đoạn sang trái thêm 1 phần tử ($currL \rightarrow currL - 1$) trong thời gian $O(1)$ thông qua hàm `add(currL - 1)`.
- Thu hẹp đoạn từ phải bớt 1 phần tử ($currR \rightarrow currR - 1$) trong thời gian $O(1)$ thông qua hàm `remove(currR)`.
- Thu hẹp đoạn từ trái bớt 1 phần tử ($currL \rightarrow currL + 1$) trong thời gian $O(1)$ thông qua hàm `remove(currL)`.

Thì từ đoạn $[currL, currR]$, ta có thể di chuyển đến bất kỳ đoạn $[L, R]$ nào của truy vấn tiếp theo bằng cách dịch chuyển hai con trỏ từng bước một.

Tổng số bước di chuyển con trỏ giữa hai đoạn $[L_1, R_1]$ và $[L_2, R_2]$ là khoảng cách Manhattan:

$$\Delta = |L_1 - L_2| + |R_1 - R_2|$$

Nếu duyệt các truy vấn theo thứ tự ngẫu nhiên trong đề bài, con trỏ có thể di chuyển $O(N)$ bước cho mỗi truy vấn, dẫn đến tổng thời gian $O(Q \cdot N)$ — quá chậm khi $N, Q \leq 2 \times 10^5$.

Thuật toán Mo (được phát minh bởi tuyển thủ Trung Quốc Mo Tao - 莫涛) là một phương pháp sắp xếp lại thứ tự của Q truy vấn ngoại tuyến (Offline Processing) một cách khéo léo dựa trên kỹ thuật **chia căn bậc hai (Square Root Decomposition)**, sao cho tổng quãng đường di chuyển của hai con trỏ L và R chỉ còn:

$$O((N + Q)\sqrt{N})$$

2. Cơ Chế Chia Khối & Chứng Minh Độ Phức Tạp

2.1. Chiến lược sắp xếp truy vấn

Ta chia mảng kích thước N thành các khối liên tiếp có kích thước cố định B :

- Khối 0: $[0, B - 1]$
- Khối 1: $[B, 2B - 1]$
- Khối k : $[k \cdot B, (k + 1) \cdot B - 1]$

Mỗi truy vấn $[L_i, R_i]$ thuộc về một khối bên trái được xác định bởi:

$$\text{block}(L_i) = \lfloor L_i / B \rfloor$$

Quy tắc sắp xếp truy vấn (Mo's Ordering):

1. Truy vấn nào có chỉ số khối $\text{block}(L)$ nhỏ hơn sẽ được xử lý trước.
2. Nếu hai truy vấn có cùng khối $\text{block}(L)$, sắp xếp theo R tăng dần.

```

struct Query {
    int l, r, id;
    int block;
    bool operator<(const Query& other) const {
        if (block != other.block)
            return block < other.block;
        return r < other.r;
    }
};

```

2.2. Chứng minh toán học nghiêm ngặt về độ phức tạp

Gọi N là kích thước mảng, Q là số lượng truy vấn, và B là kích thước mỗi khối ($1 \leq B \leq N$). Số lượng khối là xấp xỉ $\lceil N/B \rceil$.

Bước 1: Phân tích sự di chuyển của con trỏ R

- Xét tất cả các truy vấn có cùng chỉ số khối $\text{block}(L) = k$:
- Vì các truy vấn này được sắp xếp theo R tăng dần, con trỏ R chỉ di chuyển từ trái sang phải mà **không bao giờ quay lùi**.
- Do đó, trong suốt quá trình xử lý toàn bộ các truy vấn thuộc khối k , con trỏ R di chuyển từ vị trí hiện tại đến tối đa vị trí N , tổng số bước di chuyển của R trong một khối không vượt quá N .
- Vì có tối đa $\lceil N/B \rceil$ khối, khi chuyển từ khối k sang khối $k + 1$, con trỏ R có thể nhảy lùi từ cuối mảng về đầu mảng, mất tối đa N bước.
- Tổng số bước di chuyển của con trỏ R qua toàn bộ thuật toán:**

$$\text{Moves}(R) \leq \frac{N}{B} \times N = \frac{N^2}{B}$$

Bước 2: Phân tích sự di chuyển của con trỏ L

- Giữa hai truy vấn liên tiếp i và $i + 1$:
- Trường hợp A (Cùng khối $\text{block}(L_i) = \text{block}(L_{i+1})$):** Cả L_i và L_{i+1} đều nằm trọn trong cùng một khối có kích thước B . Do đó khoảng cách giữa chúng không vượt quá B :

$$|L_i - L_{i+1}| \leq B$$

- Trường hợp B (Khác khối):** Xảy ra tối đa $\frac{N}{B}$ lần, mỗi lần chuyển khối L di chuyển tối đa $2B$.
- Với Q truy vấn, tổng số bước di chuyển của con trỏ L là:

$$\text{Moves}(L) \leq Q \times B + \frac{N}{B} \times 2B = O(Q \cdot B)$$

Bước 3: Tìm kích thước khối tối ưu B

Tổng số bước di chuyển của cả hai con trở là:

$$f(B) = \text{Moves}(L) + \text{Moves}(R) = Q \cdot B + \frac{N^2}{B}$$

Áp dụng bất đẳng thức Cauchy-Schwarz (AM-GM) cho hai số dương:

$$Q \cdot B + \frac{N^2}{B} \geq 2\sqrt{Q \cdot B \cdot \frac{N^2}{B}} = 2N\sqrt{Q}$$

Dấu đẳng thức xảy ra khi và chỉ khi:

$$Q \cdot B = \frac{N^2}{B} \iff B^2 = \frac{N^2}{Q} \iff B = \frac{N}{\sqrt{Q}}$$

Kết luận:

1. Nếu $N \approx Q$: Chọn $B = \lceil \sqrt{N} \rceil$, tổng số bước là $O((N + Q)\sqrt{N}) = O(N\sqrt{N})$.
2. Nếu $Q \gg N$ (ví dụ $N = 5 \times 10^4$, $Q = 2 \times 10^5$): Việc chọn $B = \frac{N}{\sqrt{Q}}$ sẽ cân bằng chuyển dịch của L và R , mang lại tốc độ vượt trội so với việc cố định $B = \sqrt{N}$.

3. Các Kỹ Thuật Tối Ưu Hằng Số Cực Đại (Micro-optimizations)

3.1. Sắp xếp chẵn lẻ (Odd-Even Sorting)

Trong Mo tiêu chuẩn, khi kết thúc khối k (lúc này R đang ở gần cuối mảng, ví dụ $R \approx N$), sang khối $k + 1$, con trỏ R phải quay ngược từ N về lại vị trí nhỏ (ví dụ $R \approx 1$). Quãng đường quay ngược này tốn $O(N)$ bước.

Ý tưởng:

- Với khối chẵn ($\text{block}(L) \pmod{2} == 0$): Sắp xếp R tăng dần (R đi từ trái sang phải).
- Với khối lẻ ($\text{block}(L) \pmod{2} == 1$): Sắp xếp R giảm dần (R đi từ phải sang trái).

```
bool operator<(const Query& other) const {
    if (block != other.block)
        return block < other.block;
    return (block & 1) ? (r < other.r) : (r > other.r);
}
```

Hiệu quả: Con trỏ R sau khi quét đến cuối mảng ở khối k sẽ quét ngược trở lại ở khối $k + 1$ theo kiểu con lắc (Zig-Zag). Thao tác này giúp giảm chính xác một nửa tổng số bước di chuyển của R , làm chương trình chạy nhanh gấp **1.5 đến 2 lần** trên thực tế!

3.2. Sắp xếp theo Đường cong Hilbert (Hilbert Curve Ordering)

Mỗi truy vấn $[L, R]$ có thể coi là một điểm tọa độ 2 chiều (L, R) trong lưới vuông $[1 \dots N] \times [1 \dots N]$. Mục tiêu là tìm một hành trình (Hamiltonian path) đi qua tất cả Q điểm sao cho tổng khoảng cách Manhattan

$\sum(|L_i - L_{i-1}| + |R_i - R_{i-1}|)$ là nhỏ nhất.

Đường cong Hilbert là một đường cong lấp đầy không gian (Space-filling curve) có tính chất fractal tuyệt vời: hai điểm gần nhau trên đường cong 1 chiều sẽ rất gần nhau trong không gian 2 chiều.

```
inline int64_t hilbert_order(int x, int y, int pow, int rotate) {
    if (pow == 0) return 0;
    int hpow = 1 << (pow - 1);
    int seg = (x < hpow) ? ((y < hpow) ? 0 : 3) : ((y < hpow) ? 1 : 2);
    seg = (seg + rotate) & 3;
    const int rotate_delta[4] = {3, 0, 0, 1};
    int nx = x & (x ^ hpow), ny = y & (y ^ hpow);
    int nrot = (rotate + rotate_delta[seg]) & 3;
    int64_t sub_square_size = int64_t(1) << (2 * pow - 2);
    int64_t ans = seg * sub_square_size;
    int64_t add = hilbert_order(nx, ny, pow - 1, nrot);
    ans += (seg == 1 || seg == 2) ? add : (sub_square_size - add - 1);
    return ans;
}
```

Sắp xếp theo `hilbert_order(L, R)` loại bỏ hoàn toàn sự phụ thuộc vào việc tinh chỉnh kích thước khối B , cho hiệu năng tiệm cận tối ưu trên mọi tập dữ liệu truy vấn phân bố không đều.

3.3. Quy tắc thứ tự di chuyển con trỏ an toàn (Pointer Movement Order)

Khi chuyển từ $[currL, currR]$ sang $[L, R]$, thứ tự gọi 4 vòng lặp di chuyển con trỏ là **tử huyệt** để gây lỗi Run Time Error (OutOfBounds) hoặc làm sai mảng đếm tần số:

```
// ĐÚNG: Luôn MỞ RỘNG đoạn trước, rồi mới THU HẸP đoạn sau!
while (currL > q.l) add(--currL);
while (currR < q.r) add(++currR);
while (currL < q.l) remove(currL++);
while (currR > q.r) remove(currR--);
```

⚠ Cảnh báo / Bẫy:

Nếu bạn gọi `remove` trước khi `add`, trong trường hợp đoạn $[currL, currR]$ và đoạn $[q.l, q.r]$ rời nhau (ví dụ $[1, 2]$ chuyển sang $[10, 12]$), con trỏ có thể sinh ra tình trạng $currL > currR + 1$, khiến cấu trúc dữ liệu rơi vào trạng thái "đoạn âm", làm tần số bị giảm xuống giá trị âm hoặc tràn mảng!

4. Mo Có Thao Tác Cập Nhật (3D Mo with Updates)

4.1. Khái niệm và mô hình

Trong bài toán có Q thao tác xen kẽ giữa:

1. **Truy vấn đoạn:** Hỏi thông tin trên $[L, R]$.
2. **Cập nhật điểm:** Gán giá trị $A[p] = x$.

Khi giải ngoại tuyến, ta tách thành:

- Danh sách các truy vấn: $Query_i = (L, R, T, id)$, trong đó T là số lượng thao tác cập nhật đã diễn ra trước thời điểm của truy vấn này.
- Danh sách các thao tác cập nhật: $Update_t = (p, old_val, new_val)$.

Ta có một không gian trạng thái 3 chiều: $(currL, currR, currT)$.

Để chuyển giữa hai thời điểm $T_1 \rightarrow T_2$:

- Nếu $T < q.T$: Áp dụng thao tác cập nhật $T + 1$. Nếu vị trí p của update nằm trong đoạn $[currL, currR]$, ta cập nhật cấu trúc dữ liệu (`remove(old_val)` , `add(new_val)`). Sau đó $currT \leftarrow currT + 1$.
- Nếu $T > q.T$: Hoàn tác thao tác cập nhật T (gán lại `old_val`). Sau đó $currT \leftarrow currT - 1$.

4.2. Chứng minh độ phức tạp và chọn kích thước khối B

Ta sắp xếp các truy vấn 3D theo:

1. Khối của L : $block(L) = \lfloor L/B \rfloor$.
2. Khối của R : $block(R) = \lfloor R/B \rfloor$.
3. Thời điểm T tăng dần.

```
struct Query3D {
    int l, r, t, id;
    int bL, bR;
    bool operator<(const Query3D& o) const {
        if (bL != o.bL) return bL < o.bL;
        if (bR != o.bR) return bR < o.bR;
        return t < o.t;
    }
};
```

Phân tích di chuyển của 3 con trỏ:

- **Con trỏ T :** Có $(N/B) \times (N/B) = N^2/B^2$ cặp khối (bL, bR) . Trong mỗi cặp khối, T tăng đơn điệu từ 1 đến U (với $U \leq Q$ là số phép cập nhật).

$$\text{Moves}(T) \leq \frac{N^2}{B^2} \times U = O\left(\frac{N^2 Q}{B^2}\right)$$

- **Con trỏ R :** Trong cùng một khối bL , khi chuyển giữa các cặp (bL, bR) , R di chuyển tối đa B trong cùng khối hoặc N khi đổi khối bR . Tổng di chuyển của R :

$$\text{Moves}(R) \leq Q \cdot B + \frac{N}{B} \cdot N = O\left(Q \cdot B + \frac{N^2}{B}\right)$$

- **Con trỏ L :** Mỗi truy vấn, L di chuyển tối đa B :

$$\text{Moves}(L) \leq Q \cdot B$$

Tổng thời gian thực thi:

$$f(B) = O\left(Q \cdot B + \frac{N^2 Q}{B^2}\right)$$

Cân bằng hai số hạng:

$$Q \cdot B = \frac{N^2 Q}{B^2} \iff B^3 = N^2 \iff B = N^{2/3}$$

Khi chọn kích thước khối $B = N^{2/3}$, tổng độ phức tạp đạt mức tối ưu:

$$O(Q \cdot N^{2/3} + N^{2/3} \cdot N) = O(N^{5/3})$$

Với $N, Q \leq 10^5$, $N^{5/3} \approx (10^5)^{5/3} \approx 2.15 \times 10^8$ phép tính, hoàn toàn chạy tốt trong giới hạn 2-3 giây!

5. Mo Trên Cây (Mo on Trees via Euler Tour)

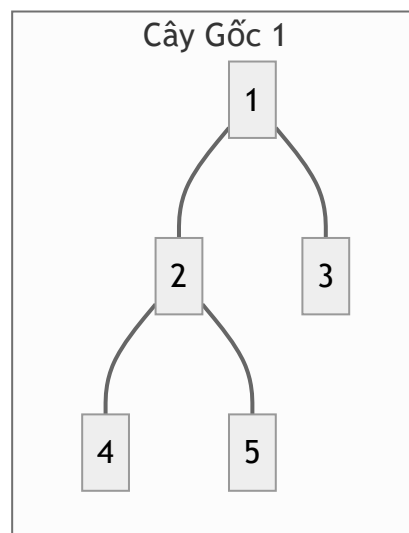
5.1. Kỹ thuật trải phẳng cây (Tree Flattening)

Cho một cây với N đỉnh. Cần trả lời Q truy vấn về đường đi đơn giữa đỉnh u và đỉnh v (ví dụ: đếm số giá trị phân biệt trên đường đi từ u đến v).

Ta trải phẳng cây thành mảng 1 chiều kích thước $2N$ bằng thứ tự thăm Euler Tour:

- Khi bắt đầu thăm đỉnh u , ta ghi nhận thời điểm vào: `tin[u] = ++timer`, và gán `euler[timer] = u`.
- Sau khi duyệt xong cây con của u , ta ghi nhận thời điểm ra: `tout[u] = ++timer`, và gán `euler[timer] = u`.

Mỗi đỉnh u xuất hiện đúng 2 lần trên mảng `euler` tại vị trí `tin[u]` và `tout[u]`.



Mảng Euler Tour tương ứng:

$$\text{euler} = [1_{in}, 2_{in}, 4_{in}, 4_{out}, 5_{in}, 5_{out}, 2_{out}, 3_{in}, 3_{out}, 1_{out}]$$

5.2. Chuyển đổi truy vấn đường đi (u, v) sang đoạn $[L, R]$

Không mất tính tổng quát, giả sử $tin[u] \leq tin[v]$. Có 2 trường hợp xảy ra:

Trường hợp 1: u là tổ tiên của v ($LCA(u, v) = u$)

- Đường đi từ u đến v tương ứng chính xác với đoạn:

$$[L, R] = [tin[u], tin[v]]$$

- Đặc điểm:** Các đỉnh nằm ngoài đường đi $u \rightarrow v$ nhưng thuộc các nhánh con khác sẽ xuất hiện **đúng 2 lần** trong đoạn này (cả `tin` và `tout`). Các đỉnh nằm trên đường đi từ u đến v chỉ xuất hiện **đúng 1 lần**.

Trường hợp 2: u không phải tổ tiên của v ($LCA(u, v) = p \neq u$)

- Đoạn cần xét trên Euler Tour là:

$$[L, R] = [tout[u], tin[v]]$$

- Đặc điểm:**
- Các đỉnh xuất hiện 1 lần trong đoạn $[tout[u], tin[v]]$ chính là các đỉnh thuộc đường đi $u \rightarrow v$, **ngoại trừ chính đỉnh $p = LCA(u, v)$!**
- Đỉnh $p = LCA(u, v)$ không xuất hiện trong đoạn $[tout[u], tin[v]]$ vì $tin[p] < tout[u]$ và $tout[p] > tin[v]$.
- Do đó, ta phải lưu thêm thông tin `lca = p` vào cấu trúc truy vấn, và tạm thời thêm p vào cấu trúc dữ liệu trước khi lấy kết quả, sau đó loại bỏ p ra.

5.3. Trạng thái bật/tắt (Toggle Node)

Ta duy trì một mảng boolean `vis[u]` (đỉnh u hiện tại có đang nằm trong tập hợp đường đi hay không):

```
void toggle(int u) {
    vis[u] ^= 1;
    if (vis[u]) {
        add(u);    // Đỉnh xuất hiện lần 1 -> Đưa vào đường đi
    } else {
        remove(u); // Đỉnh xuất hiện lần 2 -> Triệt tiêu khỏi đường đi
    }
}
```

Khi duyệt qua vị trí p trên mảng `euler`, ta chỉ cần gọi `toggle(euler[p])`. Độ phức tạp giải thuật tương đương Mô tiêu chuẩn trên mảng kích thước $2N$:

$$O((2N + Q)\sqrt{2N})$$

6. Mo Không Cần Thao Tác Xóa (Rollback Mo / Mo without Deletions)

6.1. Khi nào không thể xóa?

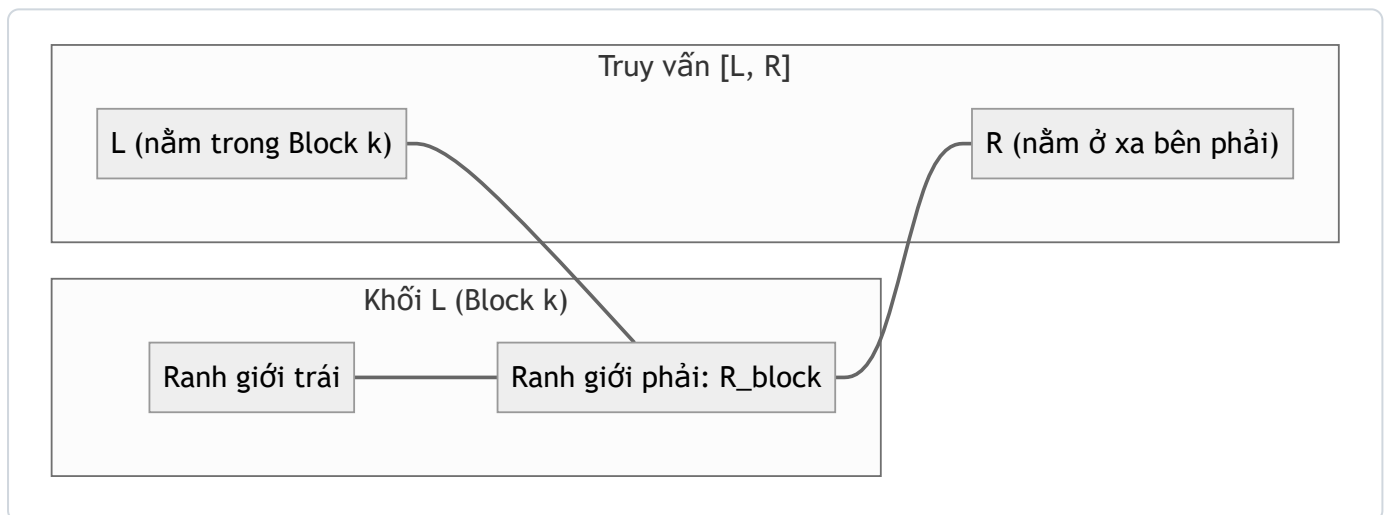
Trong nhiều cấu trúc dữ liệu, phép **thêm một phần tử (Add)** cực kỳ dễ và nhanh ($O(1)$ hoặc $O(\log N)$), nhưng phép **xóa một phần tử (Remove)** thì bất khả thi hoặc có độ phức tạp rất lớn:

1. **Disjoint Set Union (DSU)**: Khi hợp nhất hai tập hợp (Union by rank), ta chỉ cần lưu lịch sử thay đổi vào stack để hoàn tác (Rollback). Nhưng nếu muốn xóa một cạnh bất kỳ trong DSU, ta không thể làm nhanh nếu không nén đường đi.

2. **Tìm giá trị lớn nhất / nhỏ nhất (Max/Min)**: Thêm một phần tử vào để cập nhật `max` chỉ mất $O(1)$. Nhưng khi xóa phần tử `max` đi, ta không biết phần tử lớn thứ nhì là gì nếu không duy trì một cấu trúc phức tạp như Multiset ($O(\log N)$).

Rollback Mo (hay còn gọi là *Disjoint Set Mo / Mo without Deletions*) ra đời để giải quyết triệt để tình huống này.

6.2. Cơ chế hoạt động của Rollback Mo



1. Phân nhóm truy vấn:

2. Các truy vấn nằm trọn trong một khối ($R - L < B$): Trực tiếp giải bằng duyệt trâu (Brute-force) rồi reset trạng thái trong $O(B)$.

3. Các truy vấn trải qua nhiều khối: Gom nhóm theo khối của L ($\text{block}(L) = k$). Với mỗi khối k , sắp xếp các truy vấn theo R tăng dần.

4. Xử lý từng khối k :

5. Đặt ranh giới phải của khối là $R_{\text{block}} = (k + 1) \cdot B - 1$.

6. Khởi tạo lại cấu trúc dữ liệu về rỗng.

7. Đặt con trỏ $\text{curr}R = R_{\text{block}}$, $\text{curr}L = R_{\text{block}} + 1$.

8. Với mỗi truy vấn $[L_i, R_i]$:

1. **Di chuyển R**: Tăng $\text{curr}R$ dần lên đến R_i . Con trỏ R chỉ tiến, không bao giờ lùi. Các phép `add` này là **vĩnh viễn** trong khối hiện tại.

2. **Lưu vết (Snapshot)**: Ghi nhận trạng thái hiện tại của cấu trúc dữ liệu (hoặc kích thước stack hoàn tác).

3. **Di chuyển L :** Mở rộng từ R_{block} sang trái về L_i bằng các phép `add` **tạm thời**.
4. **Ghi nhận đáp án:** Lưu kết quả của truy vấn i .
5. **Hoàn tác (Rollback):** Quay lui trạng thái của con trỏ L từ L_i trở về $R_{block} + 1$, hoàn tác toàn bộ các thao tác đã thêm ở bước 3.
9. **Độ phức tạp:**
10. R di chuyển tối đa N bước cho mỗi khối $\implies$ Tổng số bước của R là $\frac{N}{B} \times N = \frac{N^2}{B}$.
11. L di chuyển tối đa B bước và rollback tối đa B bước cho mỗi truy vấn $\implies$ Tổng số bước của L là $2 \times Q \times B$.
12. Tổng độ phức tạp vẫn là $O((N + Q)\sqrt{N})$ mà **hoàn toàn không cần thao tác `remove` !**

7. Bảng Đối Chiếu Các Biến Thể Của Thuật Toán Mo

Biến thể	Kích thước khối B tối ưu	Độ phức tạp Thời gian	Độ phức tạp Bộ nhớ	Điều kiện áp dụng đặc trưng
Mo Tiêu Chuẩn (Standard Mo)	$\frac{N}{\sqrt{Q}}$	$O((N + Q)\sqrt{N})$	$O(N + Q)$	Không có update, <code>add</code> và <code>remove</code> đều $O(1)$
Mo Chẵn Lẻ (Odd-Even Mo)	$\frac{N}{\sqrt{Q}}$	$O((N + Q)\sqrt{N})$	$O(N + Q)$	Tối ưu hằng số giảm 50% số bước của R
Mo Đường Cong Hilbert	Không cần chia khối	$O((N + Q)\sqrt{N})$	$O(N + Q)$	Tối ưu hình học cực đại cho mọi phân bố query
Mo Có Cập Nhật (3D Mo)	$N^{2/3}$	$O(N^{5/3})$	$O(N + Q)$	Có truy vấn gán giá trị $A[p] = x$ xen kẽ
Mo Trên Cây (Tree Mo)	$\frac{2N}{\sqrt{Q}}$	$O((N + Q)\sqrt{N})$	$O(N + Q)$	Truy vấn đường đi trên cây, trái phải Euler Tour
Mo Không Cần Xóa (Rollback Mo)	$\frac{N}{\sqrt{Q}}$	$O((N + Q)\sqrt{N})$	$O(N + Q)$	Phép <code>add</code> khả thi nhưng <code>remove</code> bất khả thi (DSU, Max)

8. Cạm Bẫy Phổ Biến & Kinh Nghiệm Thi Đấu

1. **Sai thứ tự 4 vòng lặp di chuyển:** Luôn nhớ quy tắc **Mở rộng trước, Thu hẹp sau** (`add` trước, `remove` sau). Đảo ngược lại sẽ sinh ra đoạn rỗng có $L > R + 1$ làm âm tần số mảng hoặc tràn mảng.
2. **Kích thước khối cứng nhắc:** Không nên viết chết $B = 450$ cho mọi bài. Hãy tính động $B = \max(1, \text{int}(N/\sqrt{Q}))$. Khi $Q = 2 \times 10^5$ và $N = 5 \times 10^4$, B tối ưu xấp xỉ 112, nhanh hơn gấp đôi so với $B = \sqrt{N} \approx 223$.
3. **Mảng phân tán & Cache miss:** Tránh dùng `std::map` hoặc `std::unordered_map` bên trong hàm `add` / `remove`. Luôn nén tọa độ các giá trị về đoạn $[1 \dots N]$ từ đầu bài và sử dụng mảng tĩnh `int cnt[N]` phẳng để tối ưu bộ nhớ đệm CPU L1/L2.

4. **Mo trên cây: Quên xử lý LCA:** Khi u không phải tổ tiên của v , đoạn Euler Tour $[tout[u], tin[v]]$ không bao gồm $LCA(u, v)$. Bạn phải tạm thời `toggle(lca)` trước khi lấy kết quả và `toggle(lca)` ngay sau đó để trả lại trạng thái.

9. Tài liệu Tham khảo & Đọc thêm

1. **CP-Algorithms:**
 2. [Sqrt Decomposition & Mo's Algorithm](#) — Phân rã căn bậc hai trên truy vấn ngoại tuyến.
 3. **VNOI Wiki:**
 4. [Thuật toán Mo](#) — Kỹ thuật di chuyển 2 con trỏ và tối ưu hóa hằng số.
 5. **Codeforces Community:**
 6. [Mo's Algorithm with Updates \(3D Mo\)](#) — Không gian trạng thái 3D $O(N^{5/3})$.
 7. [An alternative sorting order for Mo's algorithm](#) — Tối ưu hóa thứ tự Hilbert Curve.
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.